```
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>  py -3.13 test_dynamic_sizes.py

======================================================================
🧪 ЗАПУСК КОМПЛЕКСНОГО ТЕСТА ДИНАМИЧЕСКИХ РАЗМЕРОВ И СКАЛЯРОВ
======================================================================
Тестирование динамических границ на устройстве: NVIDIA GeForce RTX 2060

Выполнение прямого прохода на GPU (Макс. сетка потоков: 7000)...
C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl\__init__.py:581: CompilerWarning: No
n-empty compiler output encountered. Set the environment variable PYOPENCL_COMPILER_OUTPUT=1 to see more.
  lambda: self._prg.build(options_bytes, devices),
Выполнение автоматического обратного прохода градиентов...

Верификация расчетов с оригинальным PyTorch...
  ✅ Forward (Динамические размеры): Данные X и Y полностью совпали с PyTorch!
  ✅ Backward (Защита границ): Все градиенты гетерогенных длин рассчитаны верно!

🎉 КОМПЛЕКСНЫЙ ТЕСТ ДИНАМИЧЕСКИХ ГРАНИЦ УСПЕШНО ПРОЙДЕН!
======================================================================
```

Отлично!
Правда я теперь не понимаю как мы q В компилятор отправили?

```
    init_X = np.random.uniform(-1.0, 1.0, size=(SIZE_X,)).astype(np.float32)
    init_Y = np.random.uniform(-1.0, 1.0, size=(SIZE_Y,)).astype(np.float32)

    X = Tensor.from_numpy(init_X, ctx, queue)
    Y = Tensor.from_numpy(init_Y, ctx, queue)

    X.require_grad()
    Y.require_grad()

    runner = LocalRunner(ctx, queue, is_training=True)

    # ИСПРАВЛЕНИЕ 2: Объединяем цепочки вычислений в единые монолитные вызовы,
    # чтобы JIT-компилятор построил правильные связанные уравнения Си-кода!
    # 3. ВЫПОЛНЕНИЕ ПРЯМОГО ПРОХОДА (Многострочный Текстовый JIT-Блок)
    # Передаем всю последовательность действий как единую текстовую строку!
    print(f"\nВыполнение прямого прохода на GPU (Макс. сетка потоков: {SIZE_X})...")
    runner(f"""
        X = nncl loc.sin(X)
        Y = nncl loc.sin(Y)
        X = X - q
        Y = Y - q
    """)

    # ИМИТАЦИЯ ОБРАТНОГО ПРОХОДА (Backward)
    init_grad_X_output = np.ones((SIZE_X,), dtype=np.float32)
    init_grad_Y_output = np.ones((SIZE_Y,), dtype=np.float32)

    cl.enqueue_copy(queue, X.grad_buffer, init_grad_X_output)
    cl.enqueue_copy(queue, Y.grad_buffer, init_grad_Y_output)

    # Очищаем перед тестом
    cl.enqueue_fill_buffer(queue, X.grad_buffer, np.float32(0.0), 0, X.nbytes)
    cl.enqueue_fill_buffer(queue, Y.grad_buffer, np.float32(0.0), 0, Y.nbytes)
    queue.finish()

    # Сначала зальем входящий градиент (единицы) в буферы таргетов заново,
```

раннер сейчас же текст принял?

1 небольшая оптимизация производительности

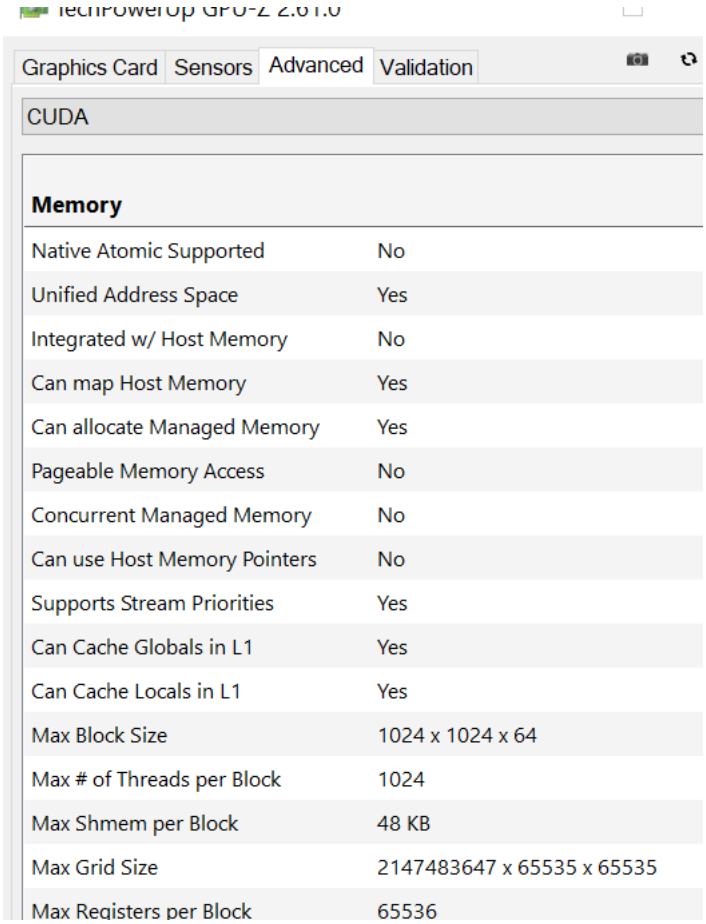
Сейчас мы вызываем всего 1 измерение.

```
        # Запускаем монолитное сплавленное ядро на GPU
        kernel(self.queue, (max_global_size,), None, *cl_buffers_call_fwd)
        self.queue.finish()

        # Стек Backward
        if self.is_training and cl_body_backward:
            backward_statements = "\n    ".join(cl_body_backward)
            backward_kernel_name = f"text_backward_{kernel_hash}"

            full_backward_code = f"""
            {BASE_CL_FUNCTIONS}
            __kernel void {backward_kernel_name}({args_str}) {{
                int tid = get_global_id(0);
                {backward_statements}
            }}
            """
            cl_buffers_backward = list(cl_buffers_forward)
```

Однако в OpenCL присутвуют как минимум 2 (а для некоторых видеокарт 3) измерения.



Поскольку мы ввели проверку размерности , мы можем ввести небольшую OpenCL оптимизацию:

TOTAL_COUNT=(element_count+element_count%32)

X_GLOBAL_SIZE=TOTAL_COUNT/32

Y_GLOBAL_SIZE=32

Тогда в ядре вычисление id будет выглядеть так:

int tid=32*get_global_size(0)+get_global_size(1)